

Magnitude, Distance and Scalar Operations on Vectors

Contents

1. Overview	2
2. Magnitude: the distance from the origin	2
2.1 In 2D and 3D	3
2.2 In n dimensions	3
3. Euclidean distance	3
3.1 Where the formula comes from	4
3.2 Distance as the norm of the difference	4
3.3 Distance in ML: the nearest neighbour	5
4. Operations with a scalar	6
4.1 Adding or subtracting a scalar: shifting	6
4.2 Multiplying or dividing by a scalar: scaling	7
5. Mean centring: shifting in ML	7
6. Summary	9
7. Key terms	9

1. Overview

Key point: Pythagoras' theorem gives both the length of a vector and the distance between two vectors, in any number of dimensions; adding or multiplying by a scalar shifts or scales a vector.

Magnitude: distance from the origin Euclidean distance between two points

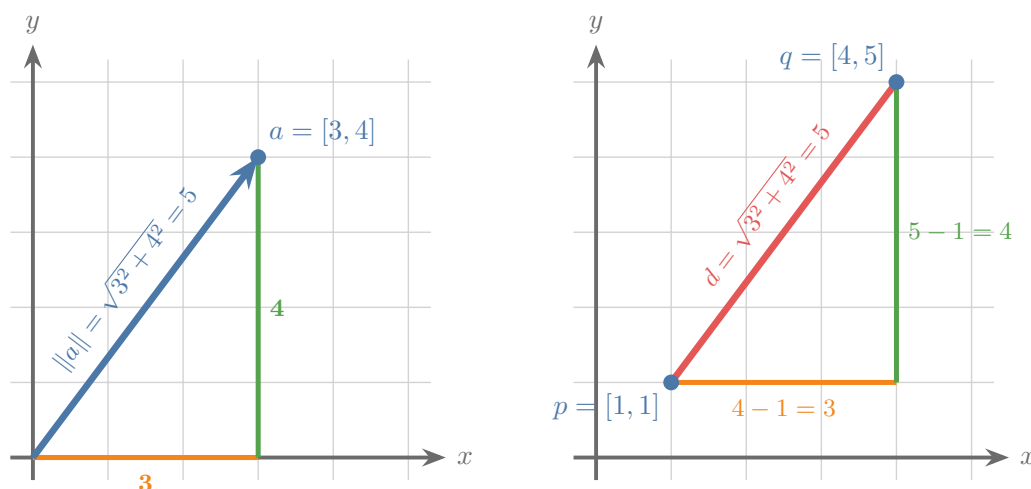


Figure 1: Left: the length of a vector. Right: the distance between two points. Both are Pythagoras' theorem

Figure 1 shows the two measurements this Note builds on. The length of $[3, 4]$ and the distance from $[1, 1]$ to $[4, 5]$ are both 5, and both come from the same right triangle. This Note works each one out in 2D, 3D and n dimensions, shows where ML uses them, and then turns to the two simplest operations on a vector: adding a scalar and multiplying by one.

Vectors, components and dimensions are defined in the [vectors and feature vectors Note](#).

2. Magnitude: the distance from the origin

Key point: The magnitude of a vector is the square root of the sum of its squared components; the same formula works in every dimension.

The **magnitude** of a vector is its distance from the origin: the length of the arrow from its tail to its head. It is also called the length or the norm of the vector (the name used in the [SVM maths Note](#)), and written $\|x\|$.

2.1 In 2D and 3D

Key point: In 2D the two components are the legs of a right triangle, so Pythagoras gives the length.

In Figure 1 (left), the vector $a = [3, 4]$ forms a right triangle with the x-axis: one leg is 3 long, the other 4. The arrow is the hypotenuse, so by Pythagoras' theorem its length is $\sqrt{3^2 + 4^2} = 5$. For a general 2D vector $[a, b]$ the length is $\sqrt{a^2 + b^2}$.

The same logic holds in 3D. A vector $[a, b, c]$ has length $\sqrt{a^2 + b^2 + c^2}$: the square root of the sum of the squares of all its components.

2.2 In n dimensions

Key point: Square every component, add, take the square root.

1. **In words:** square every component, add the squares, and take the square root.
2. **Formula:** for $x = [x_1, x_2, \dots, x_n]$,

$$\|x\| = \sqrt{x_1^2 + x_2^2 + \dots + x_n^2} = \sqrt{\sum_{i=1}^n x_i^2}$$

3. **Example:** for the 3D vector $[2, 3, 6]$,

$$\|x\| = \sqrt{4 + 9 + 36} = \sqrt{49} = 7$$

and for the 5D vector $[1, 2, 3, 4, 5]$,

$$\|x\| = \sqrt{1 + 4 + 9 + 16 + 25} = \sqrt{55} \approx 7.42$$

Python: NumPy computes the magnitude of a vector of any dimension with one function.
`import numpy as np`

```
v = np.array([1, 2, 3, 4, 5])  
np.linalg.norm(v)    # 7.416...
```

`np.linalg` is NumPy's linear algebra module. The same call works unchanged for a 15-dimensional or a 500-dimensional vector.

Extra: This is one norm among several. It is the **L2 norm**; adding the absolute values instead, $|x_1| + \dots + |x_n|$, gives the **L1 norm**. Ridge regression penalises the squared L2 norm of the coefficients and Lasso their L1 norm (see the [Ridge](#) and [Lasso](#) Notes), which is why the roadmap lists vector norms under regularisation.

3. Euclidean distance

Key point: The Euclidean distance between two vectors is the magnitude of their difference: subtract component by component, then take the norm.

The Euclidean distance, the straight-line distance between two points, is defined in the [KNN imputer Note](#) (section 4.1). Here we see where its formula comes from and how it links to the magnitude.

3.1 Where the formula comes from

Key point: The differences of the components are the legs of a right triangle; the distance is its hypotenuse.

Take two points $p = [a, b]$ and $q = [c, d]$. Draw a horizontal line from p and a vertical line from q : they meet at a right angle (Figure 1, right).

- The horizontal leg runs from a to c , so its length is $c - a$.
- The vertical leg runs from b to d , so its length is $d - b$.

By Pythagoras, the distance is $\sqrt{(c - a)^2 + (d - b)^2}$. For $p = [1, 1]$ and $q = [4, 5]$ the legs are 3 and 4 and the distance is 5. In 3D a third squared difference joins the sum, and in n dimensions there are n of them.

3.2 Distance as the norm of the difference

Key point: Subtract the two vectors, then take the magnitude of the result.

The squared differences $(c - a)^2$ and $(d - b)^2$ are exactly the squared components of the vector $q - p$. So the distance is the magnitude of the difference vector.

1. **In words:** subtract the vectors component by component, then take the magnitude of the result.
2. **Formula:**

$$d(p, q) = \|p - q\| = \sqrt{\sum_{i=1}^n (p_i - q_i)^2}$$

3. **Example:** for $p = [6, 7, 8, 9, 10]$ and $q = [1, 2, 3, 4, 5]$, the difference is $p - q = [5, 5, 5, 5, 5]$, so

$$d(p, q) = \sqrt{5 \times 5^2} = \sqrt{125} \approx 11.18$$

Python: The distance is two steps: subtract, then `norm`.

```
p = np.array([6, 7, 8, 9, 10])
q = np.array([1, 2, 3, 4, 5])
np.linalg.norm(p - q)    # 11.18...
```

This works for vectors of any dimension, as long as both have the same number of components.

3.3 Distance in ML: the nearest neighbour

Key point: Many algorithms decide by distance: a new point gets the class of the vectors nearest to it.

A lot of ML algorithms compute Euclidean distances inside. The clearest example is [K-nearest neighbours](#) (KNN), a classification algorithm. To classify a new iris flower from its sepal length and petal length, it treats the flower as a vector, computes its distance to every flower in the data, and gives it the species of the nearest ones.

The query takes the class of its nearest vector

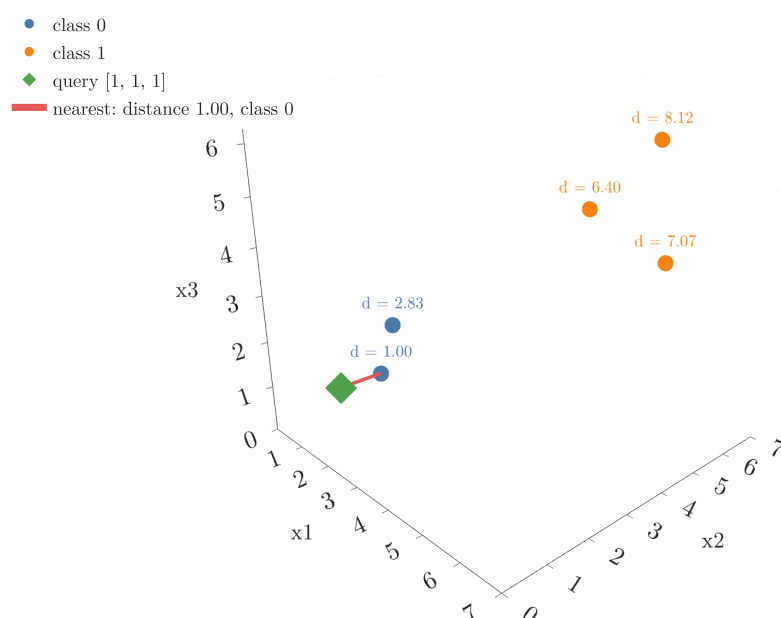


Figure 2: Five labelled vectors in 3D and a query vector; the query takes the class of the nearest one

Figure 2 runs this on five 3D vectors in two classes. The query vector $[1, 1, 1]$ is 1.00 from $[1, 2, 1]$ and at least 2.83 from all others. Its nearest neighbour is in class 0, so the query is classed 0.

The same distance appears in [K-means clustering](#), which groups points around their nearest centre, and in recommender systems, as in the [vectors and feature vectors Note](#) (section 6.3).

4. Operations with a scalar

Key point: A scalar operation applies the same number to every component: adding shifts the vector, multiplying scales it.

A vector and a scalar can be combined: the scalar is applied to every component in turn.

4.1 Adding or subtracting a scalar: shifting

Key point: Adding a scalar adds it to every component, which moves the point; this is called shifting.

1. **In words:** add the scalar to every component of the vector.
2. **Formula:** for $v = [v_1, v_2, \dots, v_n]$ and a scalar s ,

$$v + s = [v_1 + s, v_2 + s, \dots, v_n + s]$$

3. **Example:**

$$[2, 3] + 3 = [5, 6]$$

Subtraction works the same way, with s subtracted from every component: $[2, 3] - 1 = [1, 2]$.

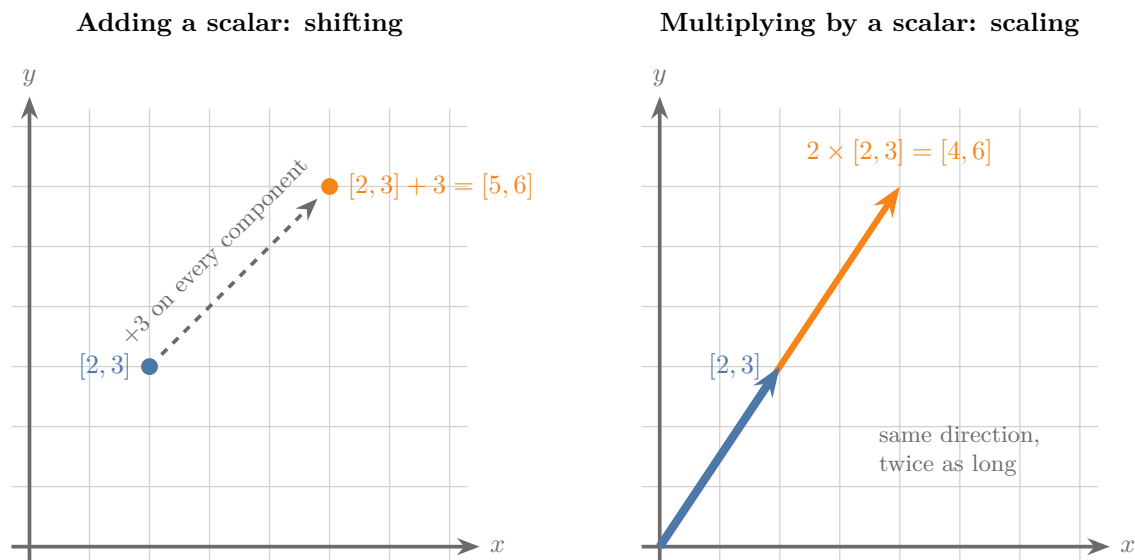


Figure 3: Left: adding a scalar moves the point. Right: multiplying by a scalar stretches the arrow

Changing every component moves the point to a new place in the coordinate system (Figure 3, left). So adding or subtracting a scalar is called **shifting**.

Extra: Strictly, adding a scalar to a vector is not an operation of linear algebra; mathematics books only define adding two vectors of the same size. NumPy allows it

through **broadcasting**: it silently stretches the scalar into the vector $[s, s, \dots, s]$ and adds that. So $[2, 3] + 3$ is really $[2, 3] + [3, 3]$.

4.2 Multiplying or dividing by a scalar: scaling

Key point: Multiplying by a scalar multiplies every component, which stretches or shrinks the arrow without turning it.

1. **In words:** multiply every component by the scalar.

2. **Formula:**

$$s v = [s v_1, s v_2, \dots, s v_n]$$

3. **Example:**

$$2 \times [2, 3] = [4, 6]$$

The result points in the same direction, but is twice as long (Figure 3, right). In general the magnitude is multiplied by $|s|$, which is why this operation is called **scaling**. Dividing by s is the same as multiplying by $1/s$: $[2, 3]/2 = [1, 1.5]$.

Extra: A negative scalar also flips the direction: $-1 \times [2, 3] = [-2, -3]$ points the opposite way. Dividing a vector by its own magnitude scales it to length 1, giving the unit vector used in the [PCA step by step Note](#): $[3, 4]/5 = [0.6, 0.8]$.

5. Mean centring: shifting in ML

Key point: Mean centring subtracts each column's mean from that column: a scalar subtraction that moves the whole cloud of points to the origin without changing its shape.

Shifting appears in ML as mean centring, the first half of [standardization](#): subtracting each column's mean so the column's mean becomes 0. Here we see it as a scalar operation on vectors.

Treat every row as a 2D vector with components x_1 and x_2 . Gather the x_1 components of all the vectors, and the x_2 components, into two columns:

	x_1	x_2	x_1 shifted	x_2 shifted
vector 1	3	4	-2	-2
vector 2	5	6	0	0
vector 3	7	8	2	2
mean	5	6	0	0

1. **In words:** compute the mean of each column, then subtract that scalar from every value of the column.

2. **Formula:**

$$x_1^{\text{shifted}} = x_1 - \bar{x}_1, \quad x_2^{\text{shifted}} = x_2 - \bar{x}_2$$

where x_1 and x_2 are whole columns (vectors) and $\bar{x}_1$, $\bar{x}_2$ are scalars.

3. **Example:** $\bar{x}_1 = (3 + 5 + 7)/3 = 15/3 = 5$ and $\bar{x}_2 = (4 + 6 + 8)/3 = 18/3 = 6$. Subtracting gives $[-2, 0, 2]$ for both columns, and each shifted column adds up to 0.

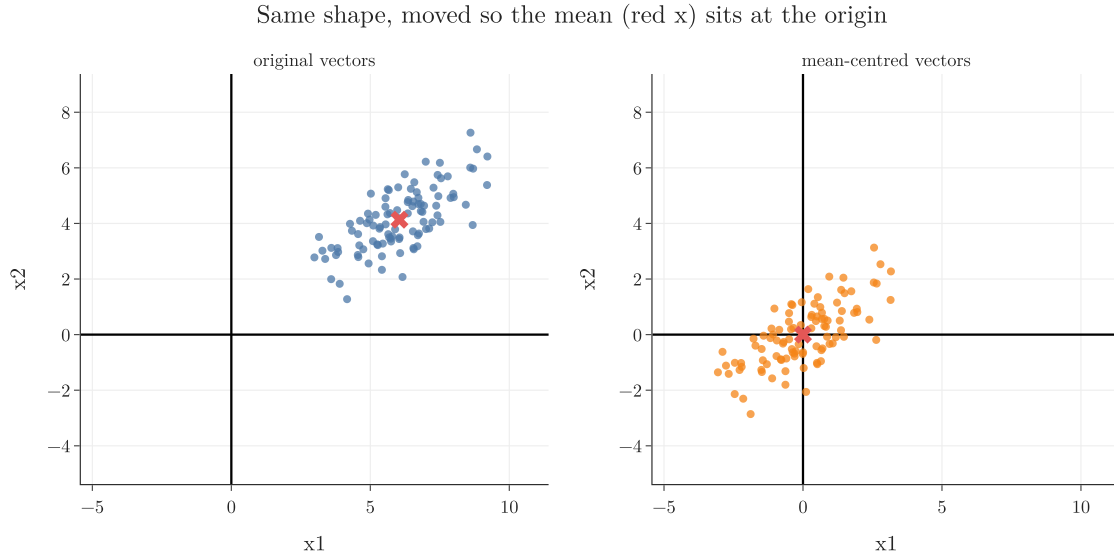


Figure 4: 100 random 2D vectors before and after mean centring

Figure 4 does the same for 100 random 2D vectors. The cloud keeps exactly its shape; only its position changes, so that its mean (the red cross) sits at the origin.

Mean centring is a useful preprocessing step. It can improve the performance, the speed of convergence and the interpretability of a model. Algorithms that use it include:

- principal component analysis, or PCA (see the [PCA intuition Note](#)),
- linear regression,
- gradient descent (see the [gradient descent Note](#)),
- clustering algorithms,
- regularisation (Ridge and Lasso).

Python: Mean centring is one line: subtracting a row of means from a table subtracts each mean from its own column.

```
data = np.array([[3, 4], [5, 6], [7, 8]])
```

```
centred = data - data.mean(axis=0) #  $[-2, -2], [0, 0], [2, 2]$ 
```

`axis=0` takes the mean down each column. The Notebook for this Note ([notebook.ipynb](#)) runs every example of this Note, including the 100-vector plot and the nearest-neighbour demo with a query you can change.

6. Summary

Idea	Formula	Example
Magnitude	$\ x\ = \sqrt{\sum x_i^2}$	$\ [3, 4]\ = 5$
Euclidean distance	$\ p - q\ $	$[1, 1]$ to $[4, 5]$: 5
Shifting	$v + s$: add s to every component	$[2, 3] + 3 = [5, 6]$
Scaling	sv : multiply every component by s	$2 \times [2, 3] = [4, 6]$
Mean centring	each column minus its mean	$3, 5, 7 \rightarrow -2, 0, 2$

- Pythagoras' theorem gives both the length of a vector and the distance between two vectors, in any dimension.
- Distance = magnitude of the difference: `np.linalg.norm(p - q)`.
- KNN, K-means and recommender systems all decide by distance.
- Shifting moves a vector, scaling stretches it; mean centring is shifting applied to whole columns.

7. Key terms

Term	Meaning
Magnitude (norm, length) of a vector	Its distance from the origin: $\sqrt{x_1^2 + \dots + x_n^2}$
L2 norm	The usual magnitude: square root of the sum of squared components
L1 norm	The sum of the absolute values of the components
Shifting	Adding or subtracting a scalar to every component of a vector
Broadcasting	NumPy stretching a scalar (or smaller array) to match a bigger array before an operation
Scaling (a vector)	Multiplying or dividing every component of a vector by a scalar